# SEARCHING & SORTING ALGORITHM

DSAA 211: WEEK 7



# SEARCHING

- SEARCHING IS WHEN WE FIND SOMETHING IN A DATA STRUCTURE.
- SEARCHING MEANS LOOKING FOR A SPECIFIC ITEM IN A COLLECTION, LIKE FINDING A WORD IN A DICTIONARY OR A SONG IN YOUR PHONE PLAYLIST.
- DEPENDING ON HOW OUR DATA IS ORGANIZED, WE CAN SEARCH IN DIFFERENT WAYS.
  - FOR UNORGANIZED DATA, WE USUALLY HAVE TO DO A LINEAR SEARCH
  - IF OUR DATA IS IN A STRICT ORDER, WE CAN PERFORM A BINARY SEARCH

# LINEAR SEARCH

- THE MOST STRAIGHTFORWARD TYPE OF SEARCH IS THE LINEAR SEARCH.
- WE TRAVERSE THE DATA STRUCTURE (E.G., A STRING'S CHARACTERS, OR A LIST) UNTIL WE FIND THE RESULT.

EXAMPLE:

WE ARE SEARCHING FOR 15

```
lst = [12, 4, 9, 18, 53, 82, 15, 99, 98, 14, 11]
```

## EXAMPLE:

### WE ARE SEARCHING FOR 15

```
lst = [12, 4, 9, 18, 53, 82, 15, 99, 98, 14, 11]
```

```
//initialization
int arr[11] = {12, 4, 9, 18, 53, 82, 15, 99, 98, 14, 11};
int n = 11;
int find = 15;
int i;
```

## EXAMPLE:

### WE ARE SEARCHING FOR 15

```
lst = [12, 4, 9, 18, 53, 82, 15, 99, 98, 14, 11]
```

```c
//traverse
    for (i = 0; i < n; i++) {
        printf("Checking value: %d\n", arr[i]);


        if (arr[i] == find) {
            printf("The array %d is in index %d.\n", find, i);
            break; //stop the loop when found
        }
    }
```

## EXAMPLE:

### WE ARE SEARCHING FOR 15

```
lst = [12, 4, 9, 18, 53, 82, 15, 99, 98, 14, 11]
```

```
if (i == n) {
        printf("Cant find %d in the array.\n", find);
    }
```

## OUTPUT:

```
Checking value: 12
Checking value: 4
Checking value: 9
Checking value: 18
Checking value: 53
Checking value: 82
Checking value: 15
The array 15 is in index 6.
```

```
Checking value: 12
Checking value: 4
Checking value: 9
Checking value: 18
Checking value: 53
Checking value: 82
Checking value: 15
Checking value: 99
Checking value: 98
Checking value: 14
Checking value: 11
Cant find 100 in the array.
```

# PRACTICE 1 : LINEAR

- WRITE A C PROGRAM THAT WILL TRAVERSE THE LIST OF STUDENT IDS, PRINT EACH ID BEING CHECKED, AND STOP IMMEDIATELY ONCE IT FINDS THE TARGET ID (2024). IF THE ID IS NOT IN THE LIST, DISPLAY A MESSAGE SAYING IT WAS NOT FOUND.

LIST OF STUDENT IDS

1011, 1022, 1503, 1789, 2024, 2450, 3001, 3255

```c
int studentIDs[8] = {1011, 1022, 1503, 1789, 2024, 2450, 3001, 3255};
int n = 8;           // number of students
int find = 2024;     // target student ID to search
int i;

for (i = 0; i < n; i++) {
    printf("Checking student ID: %d\n", studentIDs[i]);

    if (studentIDs[i] == find) {
        printf("Student with ID %d is PRESENT (found at index %d).\n", find, i);
        break;
    }
}
if (i == n) {
    printf("Student with ID %d is ABSENT.\n", find);
}
```

# PRACTICE 2 : LINEAR

- WRITE A C PROGRAM THAT SIMULATES A STORE INVENTORY SYSTEM.
  THE STORE HAS THE FOLLOWING PRODUCT CODES STORED IN AN ARRAY:

  LIST OF PRODUCT CODE : 1001, 2005, 3050, 4022, 5002, 6011, 7020, 8055, 9009, 10010

  The program should:
  1.Ask the user to input a product code.
  2.Traverse the array and display each product code while checking.
  3.If the product is found, display "Product Available" along with its index, then stop.
  4.If the product is not found after checking all items, display "Product Not Available".

```c
#include <stdio.h>
int main() {
    int productCodes[10] = {1001, 2005, 3050, 4022, 5002, 6011, 7020, 8055, 9009, 10010};
    int n = 10;
    int find;
    int i;
    printf("Enter product code to search: ");
    scanf("%d", &find);
    for (i = 0; i < n; i++) {
        printf("Checking product code: %d\n", productCodes[i]);
        if (productCodes[i] == find) {
            printf("Product with code %d is AVAILABLE (found at index %d).\n", find, i);
            break;
        }
    }
    if (i == n) {
        printf("Product with code %d is NOT AVAILABLE in inventory.\n", find);
    }

    return 0;
}
```

# BINARY SEARCH

- BINARY SEARCH IS A SEARCHING ALGORITHM USED TO FIND THE POSITION OF A TARGET ELEMENT IN A SORTED ARRAY (EITHER ASCENDING OR DESCENDING).
- IT WORKS BY REPEATEDLY DIVIDING THE SEARCH RANGE IN HALF UNTIL THE TARGET IS FOUND OR THE RANGE BECOMES EMPTY.

# HOW IT WORKS

1. Start with the sorted array.
2. Find the middle element.
3. Compare the middle element with the target:
    1. If it matches → target found.
    2. If the target is smaller → search the left half.
    3. If the target is greater → search the right half.
4. Repeat the process until the target is found or there are no more elements left.

# EXAMPLE

`[4, 9, 12, 15, 18, 25, 30]`

**Target:** 15

- **Step 1: Middle = 12 → 15 > 12 → search right half.**
- **Step 2: New middle = 18 → 15 < 18 → search left half.**
- **Step 3: Middle = 15 → Found at index 3.**

**[4, 9, 12, 15, 18, 25, 30]**

**Target:** 15

```
    int arr[7] = {4, 9, 12, 15, 18, 25, 30};
    int n = 7;           //number of elements
    int find = 15;       //target value
    int low = 0, high = n - 1, mid;
    int found = 0;
```

**[4, 9, 12, 15, 18, 25, 30]**

**Target:** 15

```
while (low <= high) { //Binary search process
    mid = (low + high) / 2;
    printf("Checking middle value: %d (index %d)\n", arr[mid], mid);
    if (arr[mid] == find) {
        printf("Found %d at index %d.\n", find, mid);
        found = 1;
        break;
    }
    else if (arr[mid] < find) {
        low = mid + 1; //search right half
    }
    else {
        high = mid - 1; //search left half
    }
}
```

- **Step 1: Middle = 12 → 15 > 12 → search right half.**
- **Step 2: New middle = 18 → 15 < 18 → search left half.**
- **Step 3: Middle = 15 → Found at index 3.**

[4, 9, 12, 15, 18, 25, 30]

**Target:** 15

```
    if (!found) {
        printf("%d not found in the array.\n", find);
    }
```

```
Checking middle value: 15 (index 3)
Found 15 at index 3.
```

**OUTPUT:**

```
Checking middle value: 15 (index 3)
Checking middle value: 25 (index 5)
Checking middle value: 30 (index 6)
60 not found in the array.
```

# PRACTICE 1 : BINARY

- A LIBRARY SYSTEM STORES A SORTED LIST OF BOOK IDS:

  LIST OF PRODUCT CODE : 101, 205, 310, 412, 523, 634, 750, 899

  The librarian wants to check if a particular book with ID 523 is available.
  Since the list is sorted, the system should use Binary Search to quickly find the book instead of checking one by one.

```c
#include <stdio.h>
int main() {
    int bookIDs[8] = {101, 205, 310, 412, 523, 634, 750, 899};
    int n = 8;
    int find = 523;
    int low = 0, high = n - 1, mid;
    int found = 0;
    while (low <= high) {
        mid = (low + high) / 2;
        printf("Checking book ID: %d (index %d)\n", bookIDs[mid], mid);
        if (bookIDs[mid] == find) {
            printf("Book with ID %d is AVAILABLE (found at index %d).\n", find, mid);
            found = 1;
            break;
        } else if (bookIDs[mid] < find) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    if (!found) {
        printf("Book with ID %d is NOT AVAILABLE in the
library.\n", find);
    }
    return 0;
}
```

# WHAT HAPPENS IF THE LIST IS NOT SORTED?

- Binary Search only works on a sorted list.
- If the array is unsorted, Binary Search may:
  - Miss the target value even if it exists.
  - Give wrong results because it assumes order when dividing the list.
  - Sometimes it may not terminate correctly if the logic gets confused (though usually it just says "not found").

# SORTING

- REFERS TO ARRANGING DATA IN A PARTICULAR FORMAT. SORTING ALGORITHM SPECIFIES THE WAY TO ARRANGE DATA IN A PARTICULAR ORDER. MOST COMMON ORDERS ARE IN NUMERICAL AND LEXICOGRAPHICAL ORDER

**Numerical Order**

- Sorting numbers based on their value (mathematical order). E.g 2, 10, 25, 100

**Lexicographical Order (Dictionary Order)**

- Sorting data as if they are words in a dictionary (character by character). E.g 10, 100, 2, 25

# SORTING IN REAL-LIFE SCENARIOS

- **TELEPHONE DIRECTORY** – THE TELEPHONE DIRECTORY STORES THE TELEPHONE NUMBER OF PEOPLE SORTED BY THEIR NAMES, SO THAT THE NAMES CAN BE SEARCHED EASILY.
- **DICTIONARY** – THE DICTIONARY STORES WORDS IN AN ALPHABETICAL ORDER SO THAT SEARCHING OF ANY WORD BECOMES EASY.

# MEMORY USAGE:

**In-place Sorting**

A sorting algorithm that does not require extra memory or uses only a small, constant amount of additional space.

- Sorting happens within the original array by swapping elements.
- Efficient in memory usage.

**Examples:**

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort (usually considered in-place with careful implementation)

# MEMORY USAGE:

**Not-in-place Sorting**

- Requires extra memory (usually proportional to the size of the array).
- Creates new arrays or lists to help with sorting.

**Examples:**

- Merge Sort (needs extra space for merging)
- Counting Sort (needs an extra counting array)
- Radix Sort

# ORDER PRESERVATION

**Stable Sorting**

- Maintains the relative order of equal elements.
- If two elements have the same value, the one that appeared first will still appear first after sorting.

**Example:**
**Bubble Sort**
**Merge Sort**
**Insertion Sort**

**Sorting by grade, then by name:**
**Before:** (Anna, 90), (Ben, 90), (Carl, 80)
**After Stable Sort:** (Anna, 90), (Ben, 90), (Carl, 80)

# ORDER PRESERVATION

**Not Stable Sorting**

- May change the order of equal elements.
- Faster in some cases but can break secondary ordering.

**Example:**
**Selection Sort**
**Heap Sort**
**Quick Sort (default implementation)**

**Sorting by grade, then by name:**
**Before:** (Anna, 90), (Ben, 90), (Carl, 80)
**After Not Stable Sort:** (Ben, 90), (Anna, 90), (Carl, 80)

# INPUT SENSITIVITY

**Adaptive Sorting**

- Takes advantage of existing order in the array.
- If the list is already partially sorted, it will run faster than worst-case time.

**Example:**
**Insertion Sort**
**Bubble Sort**

Imagine you have 10 books already arranged in order, but only 1 book is out of place.

An adaptive sorter will quickly notice and just fix that 1 book. It doesn't waste time re-checking all 10 books again and again.

# INPUT SENSITIVITY

**Non-Adaptive Sorting**

- Takes advantage of existing order in the array.
- If the list is already partially sorted, it will run faster than worst-case time.

**Example:**
**Merge Sort**
**Heap Sort**
**Selection Sort**

**Imagine you have 10 books already arranged in order.**

**A non-adaptive sorter will still take all the books, shuffle them into groups, and re-sort everything from the start. Even though they were already sorted, it wastes time.**

# DECREASING ORDER

- A SEQUENCE OF VALUE IS SAID TO BE IN DECREASING ORDER, IF THE SUCCESSIVE ELEMENT IS LESS THAN THE CURRENT ONE.

# INCREASING ORDER

- A SEQUENCE OF VALUES IS SAID TO BE IN INCREASING ORDER, IF THE SUCCESSIVE ELEMENT IS GREATER THAN THE PREVIOUS ONE

# NON-INCREASING ORDER

- A SEQUENCE OF VALUES IS SAID TO BE IN NON-INCREASING ORDER, IF THE SUCCESSIVE ELEMENT IS LESS THAN OR EQUAL TO ITS PREVIOUS ELEMENT IN THE SEQUENCE. THIS ORDER OCCURS WHEN THE SEQUENCE CONTAINS DUPLICATE VALUES.

# NON-DECREASING ORDER

- A SEQUENCE OF VALUES IS SAID TO BE IN NON-DECREASING ORDER, IF THE SUCCESSIVE ELEMENT IS GREATER THAN OR EQUAL TO ITS PREVIOUS ELEMENT IN THE SEQUENCE.

# THERE ARE MANY, MANY DIFFERENT WAYS TO SORT ELEMENTS IN A LIST. WE WILL LOOK AT THE FOLLOWING

- INSERTION SORT
- SELECTION SORT
- MERGE SORT
- QUICKSORT
- HEAP SORT